

Slide 1/76 — Cloud delivery models (IaaS / PaaS / SaaS)

1) Core idea (easy paragraph)

Cloud “delivery models” describe **how much of the computing stack you manage** versus how much the provider manages. Think of the stack from bottom to top: networking, storage, servers, virtualization, operating system, middleware/runtime, data, and application. In **IaaS**, the provider gives you the basic infrastructure (virtual servers, storage, networking), and you still manage significant parts like the OS and above. In **PaaS**, the provider also manages more of the software platform (OS, runtime, middleware), so you focus mainly on your app and its data. In **SaaS**, the provider delivers a complete application to you as a service, and you mainly configure and use it rather than build/operate it.

2) Key points

- IaaS = you manage **more** (typical: OS, middleware/runtime, app, data).
- PaaS = you manage **less** (typical: app + data; platform is managed).
- SaaS = you manage **least** (mostly user settings, not the stack).
- “Managed by vendor” usually increases as you move from IaaS → SaaS.
- Trade-off: more control (IaaS) vs more convenience (SaaS).
- PaaS is often chosen to speed development by removing OS/runtime ops work.

3) Key terms

- **Cloud stack:** Layered view of components from hardware/network up to applications.
- **IaaS:** Infrastructure service where you provision virtual infrastructure and manage OS+.
- **PaaS:** Platform service where the provider manages platform layers; you deploy code.
- **SaaS:** Finished software delivered over the network; you primarily use/configure it.
- **Virtualization layer:** Software that enables multiple isolated environments on hardware.

Slide 2/76 — Types of clouds (deployment models)

1) Core idea (easy paragraph)

Cloud “types” here refer to **deployment models**: where the infrastructure lives and who it serves. A **public cloud** is offered to many customers and is owned/operated by a provider selling cloud services. A **private cloud** is dedicated to one organization (it might be on-prem or hosted, but it’s not shared with the general public). A **hybrid cloud** combines two or more

distinct clouds (commonly public + private) and connects them so data and applications can move or interact. There can also be shared arrangements like **community/federated clouds**, where multiple organizations share infrastructure for common goals or constraints.

2) Key points

- **Memorize the definitions:** public, private, hybrid, community/federated.
- Public cloud: shared provider infrastructure; broad availability.
- Private cloud: single-organization focus; more control/policy alignment.
- Hybrid cloud: **integration** is the key—not just “we use two clouds.”
- Hybrid must enable **portability/interoperability** (data/app movement).
- Community/federated: shared by multiple orgs with shared concerns (e.g., policy, mission).

3) Key terms

- **Deployment model:** Where cloud resources are hosted and who can use them.
- **Public cloud:** Provider-owned shared cloud offered broadly to customers.
- **Private cloud:** Cloud dedicated to a single organization.
- **Hybrid cloud:** Connected combination of distinct clouds enabling portability.
- **Portability:** Ability to move apps/data across environments with minimal change.
- **Federated/community cloud:** Shared infrastructure across organizations with common needs.

Slide 3/76 — Introduction to virtualization (why it exists + main parts)

1) Core idea (easy paragraph)

Virtualization is the idea of **turning one physical machine into multiple independent execution environments**. A virtualization layer creates the illusion of “separate computers” (virtual machines) so different operating systems or applications can run concurrently on the same hardware. This is foundational for cloud computing because it improves utilization (sharing hardware efficiently) and allows flexible provisioning (creating, moving, or resizing virtual machines as needed).

2) Key points

- Virtualization **abstracts hardware** into multiple execution environments.
- The created environment is a **virtual machine (VM)**.
- Core components you must know:

- **Host** (the real underlying hardware system)
- **Hypervisor/VMM** (software that creates/runs VMs)
- **Guest** (the virtualized environment, usually running a guest OS)
- Multiple VMs can run **concurrently** on one physical host.
- Hypervisor provides an interface that looks “real” to the guest (with certain exceptions).
- Cloud relevance: virtualization enables **isolation, multi-tenancy, elastic provisioning**.

3) Key terms

- **Virtualization**: Creating multiple isolated execution environments on one machine.
- **Virtual machine (VM)**: Software-defined computer environment running OS/apps.
- **Host**: The physical system providing CPU/memory/storage/devices.
- **Guest**: The VM instance and its running OS/apps.
- **Hypervisor / VMM**: The control layer that creates, schedules, and isolates VMs.

Slide 4/76 — VM view: host vs guest + where the hypervisor sits

1) Core idea (easy paragraph)

This slide contrasts a **physical server** running an OS directly versus a **virtualized system** where a hypervisor sits between hardware and the guest environments. On a physical server, applications run on top of a single host OS. With virtualization, the hypervisor (VMM) enables multiple guest OS instances to run, each believing it has its own machine. Understanding the placement of the hypervisor helps you reason about performance, security isolation, and who controls resources.

2) Key points

- Physical case: **App → Host OS → Host machine**.
- Virtualized case: **App → Guest OS → (Guest VM) → Hypervisor/VMM → Host machine**.
- **Guest OS**: OS running *inside* a VM.
- **Host OS**: OS running on the physical machine (especially in hosted setups).
- Hypervisor/VMM: virtualization layer that manages VM creation and execution.
- The VM is the “virtual computer” container for guest OS/apps.
- Isolation comes from hypervisor-enforced separation of guests.

3) Key terms

- **Guest OS**: Operating system running inside a VM.

- **Host OS:** Operating system on the physical machine (not always present in type-1).
- **Hypervisor (VMM):** Layer managing VMs and mediating access to hardware.
- **Guest machine/VM:** The virtual computer instance created by the hypervisor.
- **Physical host:** The real hardware platform.

Slide 5/76 — What “abstracting hardware” means (CPU, memory, storage, devices)

1) Core idea (easy paragraph)

Virtualization works because the hypervisor can make hardware resources **look private and dedicated** to each VM, even though they are shared. For CPU, a VM sees virtual CPUs (vCPUs) while the hypervisor schedules real CPU time. For memory, each VM sees its own address space; the hypervisor translates or maps guest memory safely onto real memory. For storage, a VM’s “disk” can be represented by files or logical volumes on the host. For devices and I/O (network cards, GPUs, etc.), the hypervisor can emulate devices, provide optimized virtual drivers, or sometimes pass hardware through for near-native performance.

2) Key points

- CPU: VM gets **vCPUs**; hypervisor schedules physical CPU time (time-sharing idea).
- Memory: each VM sees a **private memory space**; mapping keeps guests isolated.
- Storage: VM disk often implemented as a **file/volume**; still appears as a disk to guest.
- Devices/I/O can be:
 - **Emulated** (more compatibility, potentially slower)
 - **Paravirtualized drivers** (better performance)
 - **Pass-through** (closest to native, more constraints)
- Virtualization ≠ magic: performance depends on scheduling, device strategy, overhead.

3) Key terms

- **vCPU:** Virtual CPU presented to a VM; scheduled on real CPU cores.
- **Address translation/mapping:** Mechanism to map guest memory to real memory safely.
- **Virtual disk:** Host-backed representation of storage presented as a disk to a VM.
- **Device emulation:** Simulated hardware interface for compatibility.

- **Paravirtualization:** Using virtualization-aware interfaces/drivers to reduce overhead.
- **Pass-through:** Giving a VM more direct access to a hardware device.

Slide 6/76 — Hypervisors (VMMs) and the two main types

1) Core idea (easy paragraph)

A hypervisor (also called a Virtual Machine Monitor/Manager, VMM) is the software layer that **creates and runs VMs** and separates guests from each other and from the underlying hardware. The slide introduces two major categories. A **Type 1 (bare-metal)** hypervisor runs directly on hardware, commonly used in data centers. A **Type 2 (hosted)** hypervisor runs on top of a host OS like a normal application, often used for development/testing. The distinction matters for performance, security boundaries, and operational use cases.

2) Key points

- Hypervisor/VMM is responsible for **VM creation, scheduling, isolation**.
- It separates physical hardware from the guest OSes running above it.
- **Type 1:** runs directly on hardware (common in production/data centers).
- **Type 2:** runs on a host OS (common for desktops/dev/test).
- Type choice affects overhead, manageability, and typical deployment setting.

3) Key terms

- **Hypervisor / VMM:** Control layer that runs and manages VMs.
- **Bare-metal hypervisor (Type 1):** Hypervisor installed directly on hardware.
- **Hosted hypervisor (Type 2):** Hypervisor running as software on a host OS.
- **Isolation:** Preventing one VM's behavior from impacting others.

Slide 7/76 — Why virtualization matters for cloud computing

1) Core idea (easy paragraph)

Virtualization is a core enabler of cloud computing because it allows providers to **share physical resources safely and efficiently** among many users. By running multiple isolated virtual machines on the same hardware, cloud providers can increase utilization, provision resources quickly, and move workloads when needed. This flexibility is what makes features like on-demand servers, elastic scaling, and rapid recovery possible.

2) Key points

- Virtualization enables **resource sharing** without mixing workloads.
- Isolation allows multiple customers (tenants) on the same hardware.
- Providers can **provision VMs quickly** instead of buying new servers.
- Supports **elasticity**: grow or shrink resources as demand changes.
- Enables **migration** of workloads between physical machines.
- Improves **hardware utilization** compared to one-OS-per-server.
- Cloud services depend on virtualization for automation and scale.

3) Key terms

- **Multi-tenancy**: Multiple users sharing infrastructure safely.
- **Elasticity**: Ability to scale resources up or down as needed.
- **Provisioning**: Creating and allocating computing resources.
- **Isolation**: Keeping workloads separate for security and stability.

Slide 8/76 — Benefits of virtualization

1) Core idea (easy paragraph)

This slide summarizes the **practical advantages** of virtualization. Organizations use virtualization to reduce costs, simplify management, and improve reliability. By abstracting hardware, administrators can run workloads more flexibly, recover from failures faster, and manage systems at a higher level instead of dealing with individual machines.

2) Key points

- Better **hardware utilization** (less idle capacity).
- Reduced **capital and operational costs**.
- Faster deployment of systems and services.
- Easier **backup and recovery**.
- Improved **availability** through VM migration.
- Simplified testing and development environments.
- Supports automation and centralized management.

3) Key terms

- **Utilization**: How effectively hardware resources are used.
- **Availability**: System's ability to remain operational.
- **Migration**: Moving a VM from one host to another.
- **Operational cost (OpEx)**: Ongoing cost of running systems.

Slide 9/76 — Limitations and overhead of virtualization

1) Core idea (easy paragraph)

While virtualization offers many benefits, it is not free. The hypervisor introduces **overhead**, meaning some performance loss compared to running directly on hardware. Certain workloads, especially those requiring high I/O or

specialized hardware, may be harder to virtualize efficiently. Understanding these limitations helps explain why some systems use alternative approaches such as containers or hardware pass-through.

2) Key points

- Hypervisor adds **CPU, memory, and I/O overhead**.
- Performance may be lower than bare-metal execution.
- Hardware access may be indirect or limited.
- Device emulation can increase latency.
- Pass-through improves performance but reduces flexibility.
- Not all workloads are ideal for virtualization.

3) Key terms

- **Overhead:** Extra processing cost introduced by virtualization.
- **Latency:** Delay in processing or I/O operations.
- **Bare metal:** Running directly on physical hardware.
- **Pass-through I/O:** Direct access to hardware from a VM.

Slide 10/76 — From virtualization to containers (conceptual shift)

1) Core idea (easy paragraph)

This slide introduces the motivation for **containers** as an alternative to traditional virtual machines. Instead of virtualizing hardware and running multiple guest operating systems, containers share the **same host operating system** while isolating applications at the process level. This approach reduces overhead and allows faster startup, making containers popular for modern cloud-native applications.

2) Key points

- VMs virtualize **hardware**; containers virtualize the **OS environment**.
- Containers share the host OS kernel.
- Lower overhead compared to full VMs.
- Faster startup and shutdown times.
- Smaller resource footprint.
- Well suited for microservices and rapid deployment.

3) Key terms

- **Container:** Lightweight isolated execution environment sharing a host OS.
- **Kernel:** Core part of an operating system.
- **Cloud-native:** Applications designed for scalability and automation in the cloud.
- **Microservices:** Application architecture of small, independent services.

Slide 11/76 — Virtual machines vs containers (comparison view)

1) Core idea (easy paragraph)

This slide directly compares **virtual machines (VMs)** and **containers** to highlight how they differ in structure, overhead, and use cases. VMs package an entire operating system with the application, while containers package only the application and its dependencies, relying on the host OS kernel. Understanding this comparison helps students choose the right technology for a given cloud scenario.

2) Key points

- VMs include a **guest OS**; containers do not.
- Containers share the **host OS kernel**.
- VMs are heavier and slower to start.
- Containers are lighter and start quickly.
- VMs offer stronger isolation by default.
- Containers trade some isolation for efficiency.
- Cloud environments often **use both together**.

3) Key terms

- **Guest OS**: Full operating system inside a VM.
- **Container image**: Packaged application plus dependencies.
- **Isolation**: Degree to which workloads are separated.
- **Startup time**: How quickly an environment becomes usable.

Slide 12/76 — What Docker is and why it matters

1) Core idea (easy paragraph)

Docker is a popular platform that makes it easier to **build, package, and run containers**. It provides tools and standards so applications behave the same way across development, testing, and production environments. Docker's success comes from simplifying container usage and making container-based workflows accessible to developers.

2) Key points

- Docker is a **container platform**, not just a tool.
- It standardizes how containers are built and run.
- Helps ensure consistency across environments.
- Reduces “works on my machine” problems.
- Widely used in cloud and DevOps workflows.
- Supports rapid deployment and scaling.

3) Key terms

- **Docker**: Platform for building and running containers.
- **Container runtime**: Software that executes containers.
- **DevOps**: Practices combining development and operations.

Slide 13/76 — Docker architecture (high-level)

1) Core idea (easy paragraph)

Docker uses a **client–server architecture**. The Docker client sends commands, the Docker daemon performs the work, and containers run on the host. Images define what a container contains, while registries store and distribute those images. This separation makes Docker flexible and scalable.

2) Key points

- Docker client issues commands.
- Docker daemon builds and runs containers.
- **Images** are templates for containers.
- **Containers** are running instances of images.
- Registries store and distribute images.
- Architecture supports local and remote usage.

3) Key terms

- **Docker client:** Command-line or UI interface.
- **Docker daemon:** Background service managing containers.
- **Image:** Read-only template for containers.
- **Registry:** Storage service for container images.

Slide 14/76 — Docker images and layers

1) Core idea (easy paragraph)

Docker images are built in **layers**, where each layer represents a change or instruction. Layers are read-only and can be reused across images, which saves space and speeds up builds. When a container runs, a small writable layer is added on top of the image.

2) Key points

- Images are composed of multiple layers.
- Layers are **immutable** (read-only).
- Shared layers reduce storage usage.
- Image layering speeds up rebuilds.
- Containers add a writable layer on top.
- Layer order matters for caching.

3) Key terms

- **Layer:** Individual filesystem change in an image.
- **Immutable:** Cannot be modified after creation.
- **Writable layer:** Container-specific filesystem layer.
- **Caching:** Reusing unchanged layers to save time.

Slide 15/76 — Docker Hub and image registries

1) Core idea (easy paragraph)

Docker Hub is a **public image registry** that allows users to share and retrieve container images. Registries can be public or private and are essential for distributing applications in containerized environments. Images are pulled from registries when containers are created.

2) Key points

- Docker Hub is a common **public registry**.
- Images can also be stored in private registries.
- Registries enable image sharing and reuse.
- Pulling an image downloads its layers.
- Versioning and tags identify image variants.
- Cloud workflows rely heavily on registries.

3) Key terms

- **Registry:** Service for storing container images.
- **Docker Hub:** Public container image registry.
- **Tag:** Label identifying image versions.
- **Pull:** Downloading an image from a registry.

Slide 16/76 — Containers and the host operating system

1) Core idea (easy paragraph)

Containers run **on top of the host operating system**, sharing its kernel rather than bringing their own. This design explains why containers are lightweight and fast, but also why they depend on compatibility with the host OS. The slide emphasizes that containers isolate processes, not entire machines.

2) Key points

- Containers share the **host OS kernel**.
- No separate guest OS per container.
- Isolation happens at the **process level**.
- Faster startup than VMs.
- Lower memory and storage overhead.
- Kernel compatibility matters.

3) Key terms

- **Host OS:** Operating system running on the physical or virtual host.
- **Kernel sharing:** Multiple containers using the same OS kernel.
- **Process isolation:** Separating running processes from each other.

Slide 17/76 — Namespaces: isolation in containers

1) Core idea (easy paragraph)

Namespaces are a core mechanism that containers use to **isolate resources**. Each container can have its own view of system resources such as processes, network interfaces, and file systems. Even though containers share the same kernel, namespaces make them appear independent.

2) Key points

- Namespaces limit what a process can “see.”
- Common namespaces: process, network, mount, user.
- Containers rely on namespaces for isolation.
- Each container gets its own logical environment.
- Namespaces improve security and organization.

3) Key terms

- **Namespace:** Kernel feature that isolates system resources.
- **Process namespace:** Isolates process IDs.
- **Network namespace:** Isolates network interfaces.
- **Mount namespace:** Isolates filesystem views.

Slide 18/76 — cgroups: resource control in containers

1) Core idea (easy paragraph)

While namespaces isolate what a container can see, **control groups (cgroups)** control how much it can use. Cgroups limit and account for CPU, memory, and other resources. Together, namespaces and cgroups enable safe multi-container environments on the same host.

2) Key points

- Cgroups control **resource usage**, not visibility.
- Limit CPU, memory, and I/O.
- Prevent one container from starving others.
- Enable fair resource sharing.
- Essential for multi-tenant container hosts.

3) Key terms

- **cgroups:** Kernel mechanism for resource control.
- **Resource limits:** Caps on CPU, memory, etc.
- **Accounting:** Tracking resource usage.

Slide 19/76 — Container filesystem concepts

1) Core idea (easy paragraph)

Containers use a **layered filesystem** that combines image layers with a writable layer for each container. This design allows multiple containers to

share the same base image while keeping their runtime changes separate. It improves efficiency and consistency across deployments.

2) Key points

- Base image layers are shared.
- Container adds a writable layer.
- Changes stay inside the container layer.
- Supports fast startup and low storage usage.
- Layering enables reuse across images.

3) Key terms

- **Layered filesystem:** Filesystem composed of stacked layers.
- **Writable layer:** Top layer unique to a container.
- **Base image:** Initial image layers shared across containers.

Slide 20/76 — Overlay filesystem (OverlayFS idea)

1) Core idea (easy paragraph)

Overlay-style filesystems allow multiple read-only layers to be combined with a writable layer, appearing as a single filesystem to the container. This approach is commonly used to implement container image layering efficiently.

2) Key points

- Combines multiple layers into one view.
- Read-only image layers + writable container layer.
- Efficient storage and performance.
- Widely used for container images.
- Simplifies image distribution.

3) Key terms

- **Overlay filesystem:** Filesystem that merges multiple layers.
- **Read-only layer:** Immutable image layer.
- **Copy-on-write:** Writing changes without modifying original data.

Slide 21/76 — Container networking (basic idea)

1) Core idea (easy paragraph)

Containers need networking so applications can communicate with each other and with the outside world. Container platforms provide **virtual networking** that connects containers to each other and to external networks while keeping them logically isolated. This networking is software-defined and managed by the container runtime.

2) Key points

- Containers do not automatically share the host's network.
- Each container can have its own network namespace.

- Virtual networks connect containers together.
- Containers can communicate via IP and ports.
- Port mapping exposes container services externally.
- Networking is created and managed by the platform.

3) Key terms

- **Container networking:** Virtual network connectivity for containers.
- **Network namespace:** Isolated view of networking resources.
- **Port mapping:** Exposing container ports to the host/network.

Slide 22/76 — Docker network types (high-level)

1) Core idea (easy paragraph)

Docker supports different **network modes** to meet different needs. Some modes prioritize isolation, others simplicity or performance. Understanding these modes helps you decide how containers communicate with each other and the outside world.

2) Key points

- Bridge networks are common by default.
- Host networking removes network isolation.
- None network disables networking entirely.
- Different modes trade isolation for simplicity/performance.
- Network choice affects security and accessibility.

3) Key terms

- **Bridge network:** Default isolated virtual network.
- **Host network:** Container shares host's network stack.
- **None network:** Container has no network access.

Slide 23/76 — Bridge networking explained

1) Core idea (easy paragraph)

In bridge networking, containers are connected to a **virtual bridge** that acts like a software switch. Containers can communicate with each other through the bridge, and the host can forward traffic in and out using port mappings.

2) Key points

- Virtual bridge connects containers.
- Containers get private IP addresses.
- Traffic is routed via the bridge.
- Port mapping enables external access.
- Provides isolation from host network.
- Default networking mode in many setups.

3) Key terms

- **Virtual bridge:** Software-based network switch.
- **Private IP:** IP address not directly exposed externally.
- **Port forwarding:** Redirecting traffic to a container.

Slide 24/76 — Host networking mode

1) Core idea (easy paragraph)

With host networking, a container uses the **host's network stack directly**. This removes network isolation but can improve performance and simplify certain configurations. It is useful in special cases but reduces security separation.

2) Key points

- Container shares host IP and ports.
- No network namespace isolation.
- Fewer networking layers → lower latency.
- Increased security risk.
- Port conflicts can occur.
- Typically used sparingly.

3) Key terms

- **Host networking:** Container uses host network directly.
- **Latency:** Delay in network communication.
- **Port conflict:** Two services trying to use the same port.

Slide 25/76 — None networking mode

1) Core idea (easy paragraph)

The none network mode disables networking for a container entirely. This is useful for tasks that do not require communication or for maximum isolation. The container runs but cannot send or receive network traffic.

2) Key points

- No network interface for the container.
- Maximum isolation from networks.
- Useful for batch or offline tasks.
- No inbound or outbound communication.
- Must be explicitly enabled.

3) Key terms

- **None network:** Mode with no network access.
- **Isolation:** Separation from external systems.
- **Offline workload:** Task that doesn't require networking.

Slide 26/76 — Port mapping and service exposure

1) Core idea (easy paragraph)

Port mapping allows services running **inside a container** to be accessed from outside the container. The container listens on an internal port, and the host maps that to a different (or the same) external port. This mechanism is essential for exposing web servers, APIs, or databases running in containers.

2) Key points

- Containers have their own internal ports.
- Port mapping connects host ports to container ports.
- Enables external clients to reach container services.
- Multiple containers can run the same internal port.
- Host port must be unique.
- Mapping is explicit, not automatic.

3) Key terms

- **Port mapping:** Linking host ports to container ports.
- **Internal port:** Port inside the container.
- **External port:** Port exposed on the host.

Slide 27/76 — Container lifecycle (basic states)

1) Core idea (easy paragraph)

Containers move through a **lifecycle**, from creation to termination. Understanding these states helps explain how containers are managed and why they are considered lightweight and disposable. Containers are typically created, started, stopped, and removed as needed.

2) Key points

- Container is created from an image.
- It starts and runs a process.
- It can be stopped or paused.
- Stopped containers still exist.
- Removed containers free resources.
- Containers are designed to be **ephemeral**.

3) Key terms

- **Lifecycle:** Sequence of states a container goes through.
- **Ephemeral:** Short-lived by design.
- **Stopped container:** Container that exists but isn't running.

Slide 28/76 — Containers vs virtual machines (exam comparison)

1) Core idea (easy paragraph)

This slide revisits the comparison between containers and virtual machines with an exam-oriented focus. It emphasizes differences in startup time, resource usage, isolation level, and typical use cases.

2) Key points

- Containers start faster than VMs.
- VMs provide stronger isolation.
- Containers use fewer resources.
- VMs include full OS per instance.
- Containers are ideal for microservices.
- VMs suit legacy or mixed-OS workloads.

3) Key terms

- **Startup latency:** Time to begin execution.
- **Resource footprint:** Amount of CPU/memory/storage used.
- **Legacy application:** Older software requiring full OS.

Slide 29/76 — Why containers are popular in cloud environments

1) Core idea (easy paragraph)

Containers fit well with cloud computing because they support **scalability, portability, and automation**. They allow applications to be packaged once and run consistently across environments, making them ideal for modern deployment pipelines.

2) Key points

- Portability across environments.
- Fast scaling and replication.
- Works well with automation tools.
- Smaller images reduce transfer time.
- Supports continuous deployment.
- Cloud providers integrate container support.

3) Key terms

- **Portability:** Ability to run unchanged in different environments.
- **Scalability:** Ability to handle increased workload.
- **Automation:** Programmatic management of resources.

Slide 30/76 — Introduction to container orchestration

1) Core idea (easy paragraph)

When applications use many containers, manual management becomes impractical. **Container orchestration** tools automate deployment, scaling,

networking, and recovery of containers. This slide introduces the need for orchestration rather than specific tools.

2) Key points

- Orchestration manages many containers together.
- Automates deployment and scaling.
- Handles failures and restarts.
- Manages networking and service discovery.
- Essential for production environments.

3) Key terms

- **Orchestration:** Automated coordination of containers.
- **Service discovery:** Finding services dynamically.
- **Self-healing:** Automatic recovery from failures.

Slide 31/76 — Kubernetes: why orchestration is needed

1) Core idea (easy paragraph)

This slide motivates **Kubernetes** by explaining the problems it solves. Running many containers manually is error-prone and inefficient. Kubernetes automates where containers run, how they are restarted on failure, and how they scale to meet demand. It turns containers into a manageable production platform.

2) Key points

- Manual container management does not scale.
- Kubernetes automates deployment decisions.
- Handles failures automatically.
- Manages scaling based on demand.
- Abstracts infrastructure details.
- Designed for production workloads.

3) Key terms

- **Kubernetes:** Container orchestration platform.
- **Cluster:** Group of machines managed together.
- **Automation:** System-driven management actions.

Slide 32/76 — Kubernetes cluster overview

1) Core idea (easy paragraph)

A Kubernetes cluster consists of **control components** and **worker nodes**. The control plane makes decisions and maintains the desired state, while worker nodes run the application containers. This separation allows centralized management with distributed execution.

2) Key points

- Cluster = control plane + worker nodes.
- Control plane manages state and decisions.
- Worker nodes run containers.
- Communication happens over the cluster network.
- Design supports scalability and resilience.

3) Key terms

- **Control plane:** Management components of Kubernetes.
- **Worker node:** Machine that runs application containers.
- **Desired state:** Target configuration defined by the user.

Slide 33/76 — Kubernetes control plane components

1) Core idea (easy paragraph)

The control plane includes several components that together **manage the cluster**. These components track cluster state, schedule workloads, and respond to changes. Understanding their roles helps explain how Kubernetes enforces the desired state.

2) Key points

- API server exposes the Kubernetes API.
- Scheduler decides where workloads run.
- Controller manager maintains desired state.
- Control plane components coordinate actions.
- Users interact indirectly via the API.

3) Key terms

- **API server:** Entry point for cluster management.
- **Scheduler:** Assigns workloads to nodes.
- **Controller:** Ensures actual state matches desired state.

Slide 34/76 — Worker node components

1) Core idea (easy paragraph)

Worker nodes are where application workloads actually run. Each worker includes components that manage containers, communicate with the control plane, and handle networking. These components ensure containers run as instructed.

2) Key points

- Worker nodes host containers.
- Node agent communicates with control plane.
- Container runtime executes containers.
- Networking enables pod communication.

- Nodes report status back to control plane.

3) Key terms

- **Node agent:** Component that manages node operations.
- **Container runtime:** Software that runs containers.
- **Pod networking:** Network setup for workloads.

Slide 35/76 — Pods: the basic Kubernetes unit

1) Core idea (easy paragraph)

A **Pod** is the smallest deployable unit in Kubernetes. It represents one or more containers that share networking and storage. Pods allow tightly coupled containers to run together as a single logical application component.

2) Key points

- Pod is the smallest scheduling unit.
- May contain one or multiple containers.
- Containers in a pod share IP and ports.
- Shared storage is possible within a pod.
- Pods are ephemeral by design.

3) Key terms

- **Pod:** Basic execution unit in Kubernetes.
- **Shared namespace:** Resources shared within a pod.
- **Ephemeral:** Temporary by design.

Slide 36/76 — Pods vs containers (Kubernetes context)

1) Core idea (easy paragraph)

This slide clarifies the relationship between **pods** and **containers** in Kubernetes. Containers run applications, but Kubernetes does not schedule containers directly—it schedules pods. A pod groups one or more containers that must run together and share resources.

2) Key points

- Kubernetes schedules **pods**, not containers.
- A pod may contain one or multiple containers.
- Containers in a pod share IP and ports.
- Containers in a pod often have a helper role (sidecar).
- Pods define the runtime boundary.

3) Key terms

- **Sidecar container:** Helper container in the same pod.
- **Schedulable unit:** Smallest unit Kubernetes places on a node.
- **Shared networking:** Single IP per pod.

Slide 37/76 — Kubernetes services (why they are needed)

1) Core idea (easy paragraph)

Pods are temporary and can be replaced at any time, which means their IP addresses can change. **Kubernetes Services** provide a stable way to access pods by defining a consistent endpoint. Services abstract away pod churn and enable reliable communication.

2) Key points

- Pods are ephemeral and replaceable.
- Services provide stable access points.
- Service selects pods using labels.
- Clients talk to services, not individual pods.
- Supports load balancing across pods.

3) Key terms

- **Service:** Stable abstraction for accessing pods.
- **Label:** Key–value pair used to select pods.
- **Load balancing:** Distributing traffic across pods.

Slide 38/76 — Types of Kubernetes services

1) Core idea (easy paragraph)

Kubernetes offers different service types depending on how applications should be exposed. Some services are internal to the cluster, while others allow external access. The service type determines visibility and accessibility.

2) Key points

- Internal services are cluster-only.
- External services expose apps outside the cluster.
- Choice depends on access requirements.
- Service type affects networking behavior.

3) Key terms

- **Cluster-internal service:** Accessible only inside cluster.
- **External service:** Accessible from outside the cluster.
- **Exposure:** Making an app reachable.

Slide 39/76 — Scaling in Kubernetes

1) Core idea (easy paragraph)

Scaling in Kubernetes usually means **changing the number of pod replicas**. The platform makes sure the desired number of pods are running and distributes them across nodes. This enables applications to handle changing workloads automatically or manually.

2) Key points

- Scaling adjusts number of pod replicas.
- Kubernetes maintains desired replica count.
- Supports manual and automatic scaling.
- Helps handle increased load.
- Improves availability and resilience.

3) Key terms

- **Replica:** Identical copy of a pod.
- **Scaling:** Adjusting resource quantity.
- **Availability:** Ability to handle failures.

Slide 40/76 — Self-healing behavior in Kubernetes

1) Core idea (easy paragraph)

Kubernetes continuously monitors the state of pods and nodes. When failures occur, it automatically **restarts or replaces pods** to maintain the desired state. This self-healing behavior improves reliability without manual intervention.

2) Key points

- Kubernetes detects failures.
- Automatically restarts failed pods.
- Replaces pods on unhealthy nodes.
- Works continuously in the background.
- Reduces manual recovery effort.

3) Key terms

- **Self-healing:** Automatic recovery from failure.
- **Desired state:** Target configuration defined by user.
- **Health check:** Test to detect failures.

Slide 41/76 — Kubernetes deployments (managing pods at scale)

1) Core idea (easy paragraph)

A **Deployment** is a higher-level Kubernetes object that manages pods for you. Instead of manually creating and replacing pods, you define the desired state (such as how many replicas you want), and the deployment ensures that state is maintained. Deployments make updates and rollbacks safer and more predictable.

2) Key points

- Deployments manage pod replicas.
- Define desired state declaratively.
- Automatically replace failed pods.
- Support controlled updates.
- Simplify application lifecycle management.

3) Key terms

- **Deployment:** Object that manages replicated pods.
- **Replica set:** Ensures correct number of pods.
- **Declarative configuration:** Define “what,” not “how.”

Slide 42/76 — Rolling updates and rollbacks

1) Core idea (easy paragraph)

Kubernetes deployments support **rolling updates**, which gradually replace old pods with new ones. This allows applications to be updated without downtime. If a problem occurs, Kubernetes can roll back to a previous version.

2) Key points

- Updates happen gradually.
- Old and new pods coexist temporarily.
- No downtime if configured correctly.
- Rollback restores a previous state.
- Supports safer production changes.

3) Key terms

- **Rolling update:** Gradual replacement of pods.
- **Rollback:** Reverting to a previous version.
- **Versioning:** Tracking application versions.

Slide 43/76 — Configuration and environment variables

1) Core idea (easy paragraph)

Applications often need configuration that differs between environments. Kubernetes allows configuration to be injected into pods using **environment variables or configuration objects**, keeping application code separate from environment-specific settings.

2) Key points

- Configuration is external to application code.
- Environment variables are commonly used.
- Same image can run in multiple environments.
- Improves portability and maintainability.

3) Key terms

- **Environment variable:** Key–value configuration input.
- **Configuration object:** Externalized app settings.
- **Portability:** Running unchanged across environments.

Slide 44/76 — Secrets management (high-level)

1) Core idea (easy paragraph)

Sensitive information such as passwords or tokens should not be stored directly in images or code. Kubernetes provides **secrets** to handle sensitive data securely and inject it into pods at runtime.

2) Key points

- Secrets store sensitive data.
- Injected at runtime, not baked into images.
- Reduce risk of exposure.
- Access is controlled by the platform.

3) Key terms

- **Secret:** Secure storage for sensitive data.
- **Credential:** Authentication information.
- **Access control:** Restricting who can read data.

Slide 45/76 — Storage in Kubernetes (persistent data)

1) Core idea (easy paragraph)

Containers and pods are ephemeral, but many applications need **persistent storage**. Kubernetes separates storage concerns from pods so data can survive pod restarts or replacements.

2) Key points

- Pods are temporary; data often must persist.
- Storage is abstracted from pods.
- Enables data reuse across pod lifecycles.
- Supports stateful applications.

3) Key terms

- **Persistent storage:** Data that outlives pods.
- **Volume:** Storage attached to a pod.
- **Stateful application:** App that stores data.

Slide 46/76 — Persistent volumes and claims (conceptual view)

1) Core idea (easy paragraph)

Kubernetes separates **how storage is provided** from **how it is used**. Persistent Volumes represent storage resources in the cluster, while Persistent Volume Claims are requests made by applications. This abstraction lets developers use storage without managing the underlying infrastructure.

2) Key points

- Storage is abstracted from pods.

- Persistent Volumes represent available storage.
- Claims request storage by size and type.
- Pods use claims, not volumes directly.
- Enables flexibility across environments.

3) Key terms

- **Persistent Volume (PV):** Cluster storage resource.
- **Persistent Volume Claim (PVC):** Request for storage.
- **Abstraction:** Hiding infrastructure details.

Slide 47/76 — Stateless vs stateful applications

1) Core idea (easy paragraph)

Cloud-native design distinguishes between **stateless** and **stateful** applications. Stateless apps do not store data locally and are easy to scale. Stateful apps maintain data and require careful storage handling.

2) Key points

- Stateless apps don't retain local data.
- Easy to scale and replace.
- Stateful apps need persistent storage.
- Data consistency matters for stateful apps.
- Kubernetes supports both patterns.

3) Key terms

- **Stateless application:** No local data retention.
- **Stateful application:** Maintains persistent data.
- **Data consistency:** Correctness of stored data.

Slide 48/76 — Cloud elasticity and scaling revisited

1) Core idea (easy paragraph)

Elasticity allows cloud systems to **adapt to changing demand**. Kubernetes and containers support elasticity by enabling rapid scaling of application instances based on workload needs.

2) Key points

- Elasticity adjusts resources dynamically.
- Scaling can be manual or automatic.
- Works well with stateless workloads.
- Supports cost efficiency.

3) Key terms

- **Elasticity:** Dynamic resource adjustment.
- **Autoscaling:** Automatic scaling based on metrics.
- **Demand:** Workload requirement level.

Slide 49/76 — High availability in cloud systems

1) Core idea (easy paragraph)

High availability means designing systems to **remain operational despite failures**. Cloud platforms achieve this by redundancy, replication, and automated recovery mechanisms.

2) Key points

- Failures are expected in cloud systems.
- Redundancy improves availability.
- Replication spreads workload risk.
- Automated recovery reduces downtime.

3) Key terms

- **High availability:** Minimal downtime.
- **Redundancy:** Duplicate components.
- **Replication:** Multiple copies of workloads.

Slide 50/76 — Fault tolerance vs high availability

1) Core idea (easy paragraph)

Fault tolerance and high availability are related but distinct concepts. Fault tolerance aims to **continue operation without interruption**, while high availability focuses on **minimizing downtime**.

2) Key points

- Fault tolerance masks failures completely.
- High availability allows brief disruption.
- Fault tolerance is more complex and costly.
- Cloud systems often aim for high availability.

3) Key terms

- **Fault tolerance:** No service interruption.
- **Downtime:** Period of unavailability.
- **Resilience:** Ability to recover quickly.

Slide 51/76 — Cloud cost concepts (pay-as-you-go)

1) Core idea (easy paragraph)

Cloud computing changes how organizations pay for IT resources. Instead of large upfront investments, cloud users typically pay **only for what they use**. This model aligns cost with actual demand and enables experimentation without long-term commitments.

2) Key points

- No upfront hardware purchase required.
- Costs scale with usage.

- Supports experimentation and rapid growth.
- Reduces idle resource spending.
- Enables operational expense model.

3) Key terms

- **Pay-as-you-go:** Paying based on actual usage.
- **Operational expense (OpEx):** Ongoing operational cost.
- **Capital expense (CapEx):** Large upfront investment.

Slide 52/76 — Total Cost of Ownership (TCO)

1) Core idea (easy paragraph)

Total Cost of Ownership (TCO) considers **all costs over time**, not just initial purchase. In cloud decisions, TCO includes infrastructure, maintenance, staffing, energy, and downtime considerations.

2) Key points

- TCO includes direct and indirect costs.
- Goes beyond hardware purchase price.
- Includes maintenance and operations.
- Useful for comparing cloud vs on-prem.

3) Key terms

- **TCO:** Total cost over system lifetime.
- **Maintenance:** Ongoing system upkeep.
- **Indirect cost:** Cost not directly tied to hardware.

Slide 53/76 — Cloud migration basics

1) Core idea (easy paragraph)

Cloud migration is the process of **moving applications or data** from existing environments to the cloud. Migration strategies vary depending on application design, risk tolerance, and business goals.

2) Key points

- Migration is not one-size-fits-all.
- Can involve minimal or major changes.
- Requires planning and assessment.
- Data and application dependencies matter.

3) Key terms

- **Migration:** Moving workloads to the cloud.
- **Legacy system:** Older existing system.
- **Dependency:** Relationship between components.

Slide 54/76 — Lift-and-shift migration

1) Core idea (easy paragraph)

Lift-and-shift moves applications to the cloud **with minimal changes**. The application runs largely as-is, but on cloud infrastructure. This approach is fast but may not fully exploit cloud benefits.

2) Key points

- Minimal code changes.
- Faster initial migration.
- Limited cloud optimization.
- Often uses VMs.
- Suitable for short-term goals.

3) Key terms

- **Lift-and-shift:** Move without redesign.
- **Refactoring:** Modifying application design.
- **Optimization:** Improving efficiency.

Slide 55/76 — Refactoring and re-architecting

1) Core idea (easy paragraph)

Refactoring involves **changing application design** to better use cloud features. This approach takes more effort but can improve scalability, resilience, and cost efficiency.

2) Key points

- Requires code and architecture changes.
- Enables cloud-native benefits.
- Supports containers and microservices.
- Longer migration timeline.

3) Key terms

- **Refactoring:** Changing design without changing functionality.
- **Re-architecting:** Significant redesign.
- **Cloud-native:** Designed for cloud environments.

Slide 56/76 — Choosing a migration strategy

1) Core idea (easy paragraph)

Selecting a cloud migration strategy depends on **technical, business, and risk factors**. There is no single “best” approach; instead, organizations balance speed, cost, complexity, and long-term benefits when deciding how to move workloads to the cloud.

2) Key points

- Strategy depends on application complexity.

- Business goals influence migration choice.
- Risk tolerance affects how much change is acceptable.
- Time constraints matter.
- Long-term cloud benefits vs short-term effort.

3) Key terms

- **Migration strategy:** Planned approach to moving workloads.
- **Risk tolerance:** Willingness to accept potential disruption.
- **Business driver:** Reason motivating migration.

Slide 57/76 — Cloud security responsibility model

1) Core idea (easy paragraph)

Cloud security is based on a **shared responsibility model**. Some security tasks are handled by the cloud provider, while others remain the user's responsibility. The exact division depends on the service model (IaaS, PaaS, SaaS).

2) Key points

- Security responsibility is shared.
- Provider responsibilities increase from IaaS to SaaS.
- Users still manage data and access.
- Misunderstanding roles leads to security gaps.

3) Key terms

- **Shared responsibility model:** Division of security duties.
- **Provider responsibility:** Security handled by cloud vendor.
- **Customer responsibility:** Security handled by user.

Slide 58/76 — Security in IaaS, PaaS, SaaS

1) Core idea (easy paragraph)

This slide applies the shared responsibility model to **specific cloud service models**. As you move from IaaS to SaaS, the provider takes on more security responsibility, while the user focuses more on data and access management.

2) Key points

- IaaS: user secures OS and applications.
- PaaS: provider secures platform.
- SaaS: provider secures most layers.
- Data security remains user responsibility.
- Access control is always critical.

3) Key terms

- **IaaS security:** User manages OS and apps.
- **PaaS security:** Platform secured by provider.

- **SaaS security:** Application secured by provider.
- **Access control:** Managing user permissions.

Slide 59/76 — Identity and access management (IAM) basics

1) Core idea (easy paragraph)

Identity and Access Management (IAM) controls **who can access cloud resources and what they can do**. Strong IAM practices are essential for cloud security because resources are accessible over networks.

2) Key points

- IAM manages identities and permissions.
- Least-privilege principle reduces risk.
- Authentication verifies identity.
- Authorization controls actions.
- Centralized IAM simplifies management.

3) Key terms

- **IAM:** System for managing identities and permissions.
- **Authentication:** Verifying identity.
- **Authorization:** Granting permissions.
- **Least privilege:** Minimal required access.

Slide 60/76 — Cloud compliance and governance (intro)

1) Core idea (easy paragraph)

Organizations must follow **policies, standards, and regulations** when using the cloud. Governance ensures that cloud usage aligns with organizational rules, while compliance ensures adherence to external requirements.

2) Key points

- Governance defines internal rules.
- Compliance addresses external regulations.
- Cloud usage must be monitored.
- Policies apply even in shared environments.

3) Key terms

- **Governance:** Internal policy enforcement.
- **Compliance:** Meeting external regulations.
- **Policy:** Rule guiding behavior.

Slide 61/76 — Monitoring and observability (cloud basics)

1) Core idea (easy paragraph)

Monitoring helps operators understand **what is happening inside cloud systems**. By collecting metrics, logs, and alerts, teams can detect problems

early, understand system behavior, and maintain reliability. Observability extends monitoring by helping explain *why* systems behave the way they do.

2) Key points

- Monitoring tracks system health and performance.
- Metrics show quantitative behavior (CPU, memory, etc.).
- Logs record events and messages.
- Alerts notify operators of issues.
- Observability focuses on understanding system behavior.

3) Key terms

- **Monitoring:** Tracking system state.
- **Metrics:** Numerical measurements.
- **Logs:** Recorded events.
- **Observability:** Ability to understand system behavior.

Slide 62/76 — Automation and infrastructure as code (IaC)

1) Core idea (easy paragraph)

Cloud environments are designed to be managed **programmatically**. Infrastructure as Code (IaC) treats infrastructure definitions like software, enabling repeatable, versioned, and automated deployments.

2) Key points

- Infrastructure defined in code.
- Enables repeatable deployments.
- Supports version control.
- Reduces manual configuration errors.
- Core practice in modern cloud operations.

3) Key terms

- **Infrastructure as Code (IaC):** Managing infrastructure via code.
- **Automation:** Reducing manual effort.
- **Version control:** Tracking changes over time.

Slide 63/76 — DevOps and cloud computing

1) Core idea (easy paragraph)

DevOps combines development and operations practices to **deliver software faster and more reliably**. Cloud computing supports DevOps by providing automation, scalability, and on-demand resources.

2) Key points

- DevOps improves collaboration.
- Emphasizes automation and feedback.
- Cloud enables rapid provisioning.

- Supports continuous delivery.
- Cultural and technical aspects matter.

3) Key terms

- **DevOps:** Combined development and operations practices.
- **Continuous delivery:** Frequent, reliable releases.
- **Feedback loop:** Learning from system behavior.

Slide 64/76 — Cloud reliability and resilience

1) Core idea (easy paragraph)

Reliability in cloud systems comes from **designing for failure**. Since components can fail, systems are built to recover automatically and continue operating, even under partial failure.

2) Key points

- Failures are expected.
- Redundancy improves reliability.
- Automation supports recovery.
- Resilience focuses on quick recovery.

3) Key terms

- **Reliability:** Consistent correct behavior.
- **Resilience:** Ability to recover from failure.
- **Redundancy:** Duplicate components.

Slide 65/76 — Disaster recovery concepts

1) Core idea (easy paragraph)

Disaster recovery focuses on **restoring services after major failures** such as data center outages. Cloud platforms simplify disaster recovery by enabling replication and rapid resource provisioning.

2) Key points

- Addresses large-scale failures.
- Focuses on recovery, not prevention.
- Uses backups and replication.
- Cloud enables faster recovery.

3) Key terms

- **Disaster recovery:** Restoring after major failure.
- **Backup:** Copy of data.
- **Replication:** Maintaining copies in multiple locations.

Slide 66/76 — Backup vs replication

1) Core idea (easy paragraph)

Backups and replication both protect data but serve **different purposes**. Backups are point-in-time copies used for restoration, while replication maintains near-real-time copies to support availability and recovery.

2) Key points

- Backups are periodic snapshots.
- Replication keeps data synchronized.
- Backups support long-term recovery.
- Replication supports availability.
- Both are often used together.

3) Key terms

- **Backup:** Stored copy for restoration.
- **Replication:** Ongoing data duplication.
- **Snapshot:** Point-in-time copy.

Slide 67/76 — Cloud service level agreements (SLAs)

1) Core idea (easy paragraph)

Service Level Agreements (SLAs) define **expected service performance** between providers and customers. They specify availability targets and consequences if those targets are not met.

2) Key points

- SLAs define availability expectations.
- Usually expressed as percentages.
- Include compensation or credits.
- Do not guarantee zero downtime.

3) Key terms

- **SLA:** Service Level Agreement.
- **Availability target:** Expected uptime.
- **Service credit:** Compensation for failure.

Slide 68/76 — Cloud vendor lock-in (concept)

1) Core idea (easy paragraph)

Vendor lock-in occurs when applications become **difficult to move** between providers. Proprietary services, APIs, or architectures can increase dependency on a single vendor.

2) Key points

- Lock-in limits portability.
- Proprietary services increase risk.
- Standardization reduces lock-in.
- Containers can help portability.

3) Key terms

- **Vendor lock-in:** Dependency on one provider.
- **Portability:** Ease of moving workloads.
- **Proprietary service:** Vendor-specific offering.

Slide 69/76 — Multi-cloud strategies

1) Core idea (easy paragraph)

Multi-cloud strategies use **more than one cloud provider**. Organizations may adopt multi-cloud to reduce risk, avoid lock-in, or meet regulatory needs.

2) Key points

- Uses multiple cloud providers.
- Reduces dependency on one vendor.
- Increases complexity.
- Requires consistent management practices.

3) Key terms

- **Multi-cloud:** Using multiple providers.
- **Complexity:** Increased operational effort.
- **Interoperability:** Systems working together.

Slide 70/76 — Hybrid cloud revisited

1) Core idea (easy paragraph)

Hybrid cloud combines **on-premises and cloud environments** in an integrated way. It enables workloads and data to move between environments based on policy, cost, or performance needs.

2) Key points

- Combines on-prem and cloud.
- Requires integration.
- Supports workload placement flexibility.
- Addresses regulatory or latency needs.

3) Key terms

- **Hybrid cloud:** Integrated on-prem and cloud.
- **Integration:** Connecting environments.
- **Latency:** Delay in communication.

Slide 71/76 — Cloud use cases and examples (overview)

1) Core idea (easy paragraph)

Cloud computing supports a wide range of **use cases**, from hosting websites to running data analytics and machine learning workloads. The flexibility of

cloud resources allows organizations to choose suitable services for different needs.

2) Key points

- Supports diverse workloads.
- Enables rapid experimentation.
- Scales with demand.
- Suitable for startups and enterprises.

3) Key terms

- **Use case:** Specific application scenario.
- **Workload:** Type of computing task.
- **Scalability:** Ability to handle growth.

Slide 72/76 — Cloud adoption challenges

1) Core idea (easy paragraph)

Despite its benefits, cloud adoption introduces **technical and organizational challenges**. Skills gaps, security concerns, cost management, and governance must be addressed for successful adoption.

2) Key points

- Skills and training are required.
- Security remains a concern.
- Cost management is critical.
- Governance must be established.

3) Key terms

- **Adoption:** Moving to cloud usage.
- **Skills gap:** Lack of expertise.
- **Cost management:** Controlling expenses.

Slide 73/76 — Cloud best practices (high-level)

1) Core idea (easy paragraph)

Cloud best practices help organizations **use cloud resources effectively and safely**. These practices focus on security, cost efficiency, reliability, and operational excellence.

2) Key points

- Design for failure.
- Apply least-privilege access.
- Monitor and optimize costs.
- Automate wherever possible.

3) Key terms

- **Best practice:** Recommended approach.
- **Least privilege:** Minimal access needed.
- **Automation:** Programmatic management.

Slide 74/76 — Cloud trends and future directions

1) Core idea (easy paragraph)

Cloud computing continues to evolve, with trends such as increased automation, managed services, and deeper integration with application development. Understanding trends helps anticipate future cloud usage patterns.

2) Key points

- Growth of managed services.
- Increased automation.
- Focus on developer productivity.
- Continued container adoption.

3) Key terms

- **Managed service:** Provider-managed offering.
- **Automation:** Reduced manual work.
- **Productivity:** Efficiency of development.

Slide 75/76 — Review and integration of concepts

1) Core idea (easy paragraph)

This slide ties together the main cloud concepts: service models, virtualization, containers, orchestration, security, and cost. It reinforces how these ideas interact to form complete cloud systems.

2) Key points

- Cloud models define responsibility.
- Virtualization and containers enable flexibility.
- Orchestration manages scale.
- Security and cost are shared concerns.

3) Key terms

- **Integration:** Working together of components.
- **Cloud model:** Service or deployment model.
- **Orchestration:** Coordinated management.

Slide 76/76 — Final summary and exam focus

1) Core idea (easy paragraph)

The final slide summarizes the **core learning outcomes** of the course. Students should understand cloud fundamentals, compare service and deployment models, and reason about design and migration decisions using clear constraints.

2) Key points

- Understand cloud fundamentals.
- Compare IaaS, PaaS, SaaS.
- Explain virtualization vs containers.
- Reason about migration strategies.
- Apply security responsibility concepts.

3) Key terms

- **Cloud computing:** On-demand shared resources.
- **Service model:** IaaS, PaaS, SaaS.
- **Deployment model:** Public, private, hybrid.